

Project Plan

Step 1: Goal

Why am i making this project?

A: Main project for college

Who is this project for?

A: Students and Teachers

What is gonna make it valuable?

A: It becomes valuable because it centralizes academic management and enhances learning efficiency through automation and AI support. Teachers can systematically manage deadlines, assignments, and evaluations in one platform, reducing manual tracking. Students receive timely reminders, structured academic updates, and personalized support through an AI chatbot that understands their semester, syllabus, and deadlines. This improves organization, reduces missed submissions, enhances exam preparation, and strengthens communication between students and teachers.

Step 2: User stories

Students:

- Create account
- View dashboard
- Get reminders
- Set reminders
- Use chat bot for studies, study plans, creating reminders, clearing doubts, etc.
- Attendance, internal marks, and keep track of submissions.
- LMS(Learning Management system)

Faculties:

- Create account
- View dashboard
- Set reminders/deadlines for students
- Upload internal marks
- Upload attendance
- Analyse students' performance based on attendance, internal marks
- Attendance, internal marks, keep track of submissions.

Step 3: Define data models

Core Authentication:(User)

- id (PK)
- name
- email (unique)
- password_hash
- role (Student / Faculty / Admin)
- created_at
- is_active

Academic Structure:

Department

- id (PK)
- name

Batch

(Example: 2024–2027 AI & ML)

- id (PK)
- department_id (FK → Department)
- start_year
- end_year

Semester

- id (PK)
- semester_number (1–8)
- batch_id (FK → Batch)
- academic_year (e.g., 2025–2026)
- is_active

Student Model :Improved for Manual Promotion(Student)

Instead of only storing current_semester_id, we track semester history.

- id (PK)
- user_id (FK → User)
- register_number
- batch_id (FK → Batch)
- section
- current_semester_id (FK → Semester)

StudentSemesterHistory

(VERY IMPORTANT for manual updates)

- id (PK)
- student_id (FK → Student)
- semester_id (FK → Semester)
- promoted_by (FK → Admin User)
- promotion_date
- status (Active / Completed / Failed)

This prevents data loss when the semester changes.

Faculty Model:(Faculty)

- id (PK)
- user_id (FK → User)
- department_id (FK → Department)
- designation

Subject Structure:(Subject)

- id (PK)
- subject_name
- subject_code
- department_id (FK → Department)

SubjectOffering

This connects subject + semester + faculty

- id (PK)
- subject_id (FK → Subject)
- semester_id (FK → Semester)
- faculty_id (FK → Faculty)

Enrollment:(Enrollment)

- id (PK)
- student_id (FK → Student)
- subject_offering_id (FK → SubjectOffering)
- enrollment_date

Attendance:(Attendance)

- id (PK)
- enrollment_id (FK → Enrollment)
- date
- status (Present / Absent)

Internal Evaluation:(InternalExam)

- id (PK)
- subject_offering_id (FK)
- exam_name
- max_marks
- exam_date

InternalMark

- id (PK)
- internal_exam_id (FK)
- student_id (FK)
- marks_obtained

Assignments:(Assignment)

- id (PK)
- subject_offering_id (FK)
- title
- description
- deadline
- created_at

Submission

- id (PK)
- assignment_id (FK)
- student_id (FK)
- file_url
- submitted_at
- marks
- feedback

Reminder System:(Reminder)

- id (PK)
- user_id (FK)
- title
- description
- reminder_time
- related_assignment_id (nullable FK)
- is_completed

RAG System :PDF Based(Document)

- id (PK)
- subject_offering_id (FK)
- title
- file_url
- document_type (Syllabus / Notes / Question Bank)
- uploaded_by (FK → Faculty/Admin)
- uploaded_at

DocumentChunk

- id (PK)
- document_id (FK)
- chunk_text
- embedding_vector
- chunk_index

Chatbot Tracking:(ChatSession)

- id (PK)
- user_id (FK)
- started_at

ChatMessage

- id (PK)
- session_id (FK)
- role (User / Assistant)
- message_text
- timestamp

Step 4: Nail an MVP

1. Authentication System

- Login
- Role-based access (Student / Faculty / Admin)

Without this → system doesn't function.

2. Academic Structure (Basic Version)

Keep:

- Department
- Batch
- Semester
- Student
- Faculty
- SubjectOffering

Cut:

- StudentSemesterHistory (can add later).

For MVP, you can update the `current_semester_id` manually without tracking history.

3. Faculty Core Features

Keep:

- Create assignment
- Set deadline
- Upload internal marks
- View list of students

Cut:

- Advanced performance analytics
- Graphs
- AI performance predictions

4. Student Core Features

Keep:

- View assignments
- View deadlines
- View internal marks
- Submit assignment

Cut:

- Attendance tracking (add later)
- Advanced dashboard analytics

Attendance is not essential for MVP.

5. Reminder System (Simple Version)s

Keep:

- Auto reminder for upcoming deadlines

Cut:

- Custom user reminders
- Reminder editing logic

Just:

- If the deadline is near → show a reminder.

That's enough.

6. AI Chatbot (Minimal RAG Version)

1. RAG-Based Doubt Solver

- Uses pre-uploaded syllabus/notes (by developers)
- Retrieves relevant chunks
- Generates answers using LLM

User:

- Student asks questions
- Gets subject-aware answer

2. AI Deadline Awareness

AI can:

- Check upcoming assignments
- Warn the students about deadlines
- Mention priority subjects

Data used:

- Assignment
- Deadline

3. AI Performance Awareness (Basic Version)

AI can:

- Analyze internal marks
- Detect low-scoring subjects
- Suggest focus areas

Simple rule logic + LLM explanation.

What we cutted for now?

- Attendance tracking
- Semester history tracking
- Advanced analytics
- Student performance comparison
- Study planner automation
- Multi-session chat memory
- Notification system (email/SMS)
- Role like Parent access
- Admin dashboard analytics

MVP Feature List

Admin:

- Create Department
- Create Batch
- Create Semester
- Add Students
- Add Faculty

Faculty:

- Create SubjectOffering
- Upload syllabus PDF
- Create assignments
- Set deadlines
- Upload internal marks

Student:

- Login
- View subjects
- View assignments
- Submit assignments
- View internal marks
- Ask subject-related doubts (RAG)
- Ask for exam preparation help
- Ask: "What should I study first?"
- Ask: "Am I weak in any subject?"
- Get deadline reminders via AI

Step 5: Draw a simple prototype

On Python rough record

Step 6: Understanding future of my project going to look like.

for now its college project, but in future i hope, it will be a startup project.

Step 7: Drill into specific components.

Frontend



FastAPI Backend (API Layer)



Database + Vector Store



LLM (External API)

Frontend Layer

Purpose:

User interaction.

Technology Options:

- Simple HTML + JS (for MVP)
- Or React (optional future)

Responsibilities:

- Login / Authentication
- Student dashboard
- Faculty dashboard
- Chat interface
- Assignment submission
- Display marks & deadlines

Important:

Frontend does NOT contain AI logic.

It only calls backend APIs.

Backend Layer (FastAPI)

This is the core engine.

Responsibilities:

1. Authentication (JWT)
2. Role-based access control
3. CRUD operations (assignment, marks, subjects)
4. Deadline logic
5. AI orchestration
6. API endpoints

Structure example:

- routers/ → API endpoints
- models/ → SQLAlchemy models
- schemas/ → Pydantic validation
- services/ → Business logic
- rag/ → AI pipeline
- utils/ → Helper functions

Database Layer (Relational Database)

PostgreSQL (production)

Stores:

- Users
- Students
- Faculty
- SubjectOffering
- Assignments
- InternalMarks

Purpose:

Structured academic data storage.

Vector Store (AI Memory Layer)

Purpose:

Store embeddings of syllabus PDFs.

Options:

- FAISS (local, simple)

Stores:

- Document chunks
- Embedding vectors

Used only for:

RAG retrieval.

LLM Integration Layer

Purpose:

Generate intelligent responses.

Workflow:

1. User sends query
2. Backend retrieves relevant document chunks
3. Context + query sent to LLM
4. LLM generates answers
5. Backend returns response

This is an AI brain.

Core Essential Components (Must Work for MVP)

1. Authentication Module:

- JWT token generation
- Role validation middleware

Without this → system is insecure.

2. Academic Data Module:

- SubjectOffering
- Assignments
- Marks

Without this → AI has no structured context.

3. RAG Pipeline:

- This includes:
- PDF chunking (done by developers)
- Embedding generation
- Vector storage
- Similarity search
- Context assembly
- LLM response

This is your most critical technical component.

4. AI Service Layer:

Contains:

- Performance analysis logic
- Deadline awareness logic
- Prompt engineering
- Response formatting

Separating this from routers shows architectural maturity.

5. API Gateway (FastAPI Routes):

Example endpoints:

- POST /login
- GET /assignments
- POST /submit-assignment
- POST /chat

These connect frontend and backend.

Request Flow Example

How does a student query get processed?”

Answer:

1. Student sends a question through the frontend.
2. Request hits FastAPI chat endpoint.
3. Backend queries the vector database for relevant chunks.
4. Retrieved context is combined with the user query.
5. LLM generates a response.
6. Response returned to the frontend.

Step 8: Picking your stack.

1. Backend

FastAPI

2. Database

PostgreSQL

3. ORM

SQLAlchemy

4. AI / RAG Stack

Embeddings:

- HuggingFace sentence-transformers

Vector Store:

- FAISS

4. LLM

Ollama - phi2 or phi3:mini

5. Frontend

HTML + CSS + Vanilla JS

7. Authentication

JWT Authentication

FastAPI +:

- python-jose
- passlib (for hashing)

Deployment Path

Backend Deployment

- Render.com
- Railway.app

Database Deployment

- Render PostgreSQL
- Railway PostgreSQL

Vector Store

- Store FAISS index as a local file.

In deployment:

- Keep FAISS index file in the server storage

Step 9: Overall development process

Create a project skeleton:

- Basic folder structure
- Development environments
- Version control

Setting up the database and creating data models:

Build backend routes:

- Api end Points

Frontend interface:

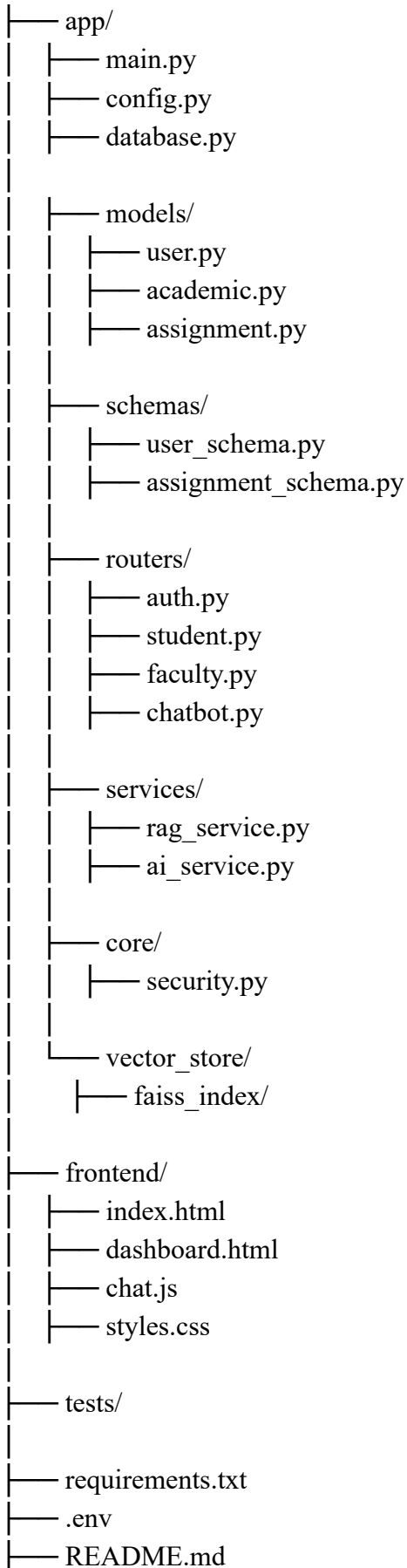
- Connect to backend
- UI/UX

Project iteration:

- Features
- Test
- Deploy early
- Automated deployments and testing

Project Skeleton

ai-academic-assistant/



Development Environment

Environment Setup

Use:

- Python 3.10+
- Virtual environment (venv)
- pip for dependency management

Commands:

```
python -m venv venv
```

```
source venv/bin/activate
```

```
pip install fastapi uvicorn sqlalchemy psycpg2-binary
```

For AI:

```
pip install openai faiss-cpu langchain
```

Version Control

Use Git

Initialize:

```
git init
```

Create .gitignore:

- venv/
- pycache/
- .env
- faiss_index/

Push to GitHub early.

Commit structure:

- Initial skeleton
- Auth module
- Database models
- API routes
- AI integration
- Deployment config

Commit often.

Database Setup & Models

Step 1: Connect Database:

In database.py:

- Create engine
- Create session
- Base = declarative_base()

Step 2: Create Models:

Order:

1. User
2. Student
3. Faculty
4. SubjectOffering
5. Assignment
6. InternalMark

After models:

`Base.metadata.create_all(bind=engine)`

Test database before building routes.

Build Backend Routes

Phase 1: Authentication Routes

Endpoints:

- POST /register
- POST /login

Test with Swagger UI.

Phase 2: Academic Routes

Student:

GET /student/assignments

GET /student/marks

Faculty:

POST /faculty/assignment

POST /faculty/marks

Test everything via Swagger first.

Phase 3: AI Routes

POST /chat

Flow:

1. Receive query
2. Call the RAG service
3. Retrieve chunks
4. Call LLM
5. Return response

Test AI separately before connecting frontend.

Frontend Interface

Step 1: Login Page

- Fetch POST /login
- Store JWT in localStorage

Step 2: Dashboard Page

On load:

- Fetch assignments
- Fetch marks
- Display in simple table

Step 3: Chat Interface

When user sends message:

- POST /chat with JWT
- Display response

Keep minimal UI.

Project Iteration Strategy

Iteration Rule

Never build everything at once.

Follow this loop:

Build → Test → Improve → Commit

Milestone 1

Auth + DB + Swagger working.

Deploy early.

Milestone 2

Assignments + Marks working.

Deploy again.

Milestone 3

RAG working locally.

Milestone 4

AI Chat integrated.

Deployment Early Strategy

Deploy backend to:

Render.com

Steps:

1. Connect GitHub repo
2. Add environment variables
3. Attach PostgreSQL
4. Deploy

Test API live before frontend.

Automated Deployment

Use:

- GitHub repository
- Render auto-deploy on push

Whenever you push:

Server redeploys automatically.

Basic Testing Strategy

Just:

- Use FastAPI Swagger
- Write simple test scripts
- Manually test AI responses

Optional:

pytest for basic API testing.

Clean Development Timeline

Week 1:

Project skeleton + DB + Auth

Week 2:

Academic routes

Week 3:

RAG integration

Week 4:

Frontend + Deployment